Draft version

Robust approach for classifying time series from dynamical systems

Kirill Semkin[1*] and Vadim Strijov[2*]

[1*]MIPT Moscow Russia.
[2*]MIPT Moscow Russia.

*Corresponding author(s). E-mail(s): semkin.ki32@gmail.com; strijov@forecsys.ru;

Keywords: time series; classification; dynamical systems; NeuralODE; inertial measurement unit

1 Introduction

The paper introduces and analyzes methods for time series classification. The *time series* is a sequence of scalar values $x(t_i) = \{x(t_1), \ldots, x(t_n)\}$ defined in discrete times $t_1, \ldots, t_n$. The *multivariate* time series is a sequence of vector values $\mathbf{x}(t_i) = \{\mathbf{x}(t_1), \ldots, \mathbf{x}(t_n)\}$ where values' dimension is $\mathbb{R}^s$. This is equal to having s time series with the same time domain. Everywhere further we assume time series to be multivariate in general. In general, time series values $\mathbf{x}(t_i)$ are *random variables*. We assume them to decompose into deterministic component $\mathbf{d}(t_i)$ and random noise component $\boldsymbol{\epsilon}(t_i)$ so $\mathbf{x}(t_i) = \mathbf{d}(t_i) + \boldsymbol{\epsilon}(t_i)$. Next, any time series has an associated *class label* $y \in 1 \dots K$. Here K is the number of classes. A *classifier* is a function $\delta(\tilde{\mathbf{x}}(t_i))$ that associates any time series with a class label. To measure classification quality we introduce a *risk function* $r(\delta)$. Now, formulate time series classification problem as:

Problem 1. *Given a set of N labeled time series $\mathcal{D} = \{(\tilde{\mathbf{x}}_1(t_i), y_1), \dots, (\tilde{\mathbf{x}}_N(t_i), y_N)\}$ solve an optimization problem $r(\delta) \to \min$. The noise terms $\{\boldsymbol{\epsilon}_1(t_i), \dots, \boldsymbol{\epsilon}_N(t_i)\}$ are assumed to be mutually independent processes.*

There are many methods for time series classification in the literature. *Distance-based* approaches rely on measuring similarity between time series using distance metrics. The metric can be defined using Dynamic Time Warping [1] and the k-Nearest-Neighbors (k-NN) classifier can be used then [2]. *Feature-based* approaches extract finite set of statistical [3, 4] or spectral [5] features from time series for further classification. *Deep learning* approaches uses neural networks to automatically learn time series features from raw train data. Convolutional neural networks [6, 7], recurrent neural networks (RNN) [8] or their combination [9–11] are designed to process sequential data like time series.

To solve the classification problem 1 we need to clarify the rule of assigning class labels to the time series. We also need to introduce a model of the time series. In the paper we assume this model to be a *dynamical system*. We define it as a time-invariant *ordinary differential equation (ODE)* of the form

$$\begin{cases} \frac{d}{dt}\mathbf{z}(t) = f(\mathbf{z}), \\ \mathbf{z}(0) = \mathbf{z}_0, \end{cases} \tag{1}$$

where $\mathbf{z} \in M$, $f(\mathbf{z}) : M \to M$ is a *vector field*, $\mathbf{z}_0$ is a *starting point*, M is an m-dimensional differentiable manifold. To find an $\mathbf{z}(t)$ satisfying (1) on some time interval is to solve an *initial value problem*. Under some regularity conditions on f the solution is unique [12]. The $\mathbf{z}(t)$ is called *phase trajectory*. To connect the dynamical system with the time series we introduce an *observation map* $\alpha : M \to \mathbb{R}^s$. We say a time series $\mathbf{x}(t_i)$ is generated by the phase trajectory $\mathbf{z}(t)$ if $\mathbf{x}(t_i) = \alpha(\mathbf{z}(t_i))$ for all t_i. Further we assume all time series to be generated. Next, consider K different dynamical systems with the vector fields $f_j,\ j \in 1, \dots, K$. If a time series is generated by a phase trajectory from the system j, the time series' label is $y = j$. Hence, we have completely described the time series model. Note that $\mathbf{z}(t_i)$ are not observed and therefore are not part of the $\mathcal{D}$.

Our time series' model can be applied to many real-word classification problems. For example, in medicine an electroencephalogram (EEG) is a direct, macroscopic measure of the brain's electrical activity [13]. The activity is governed by the Maxwell's Equations that can be reduced to the ODEs [14]. The EEG can be used for detecting arrhythmias or other heart conditions [6] as well as to identify seizures and sleep stages from brainwave data [7]. In the weather forecasting the fundamental equations of atmospheric dynamics can also be reduced to an ODEs system [15]. The classification problems in this field include weather regime classification [16] and climate pattern identification [17]. In human activity recognition (HAR) ODEs are primarily used to model the dynamics of human motion. The core idea is that human movement is a continuous physical process governed by laws of mechanics, which can be described by differential equations [18]. This idea can be applied to the basic or fine-grained activity recognition [19, 20], fall detection [21], and other tasks. To conclude, our model is applicable when it is reasonable to assume that observed time series come from some dynamical system of the form (1).

To solve the problem 1 we first *reconstruct* the phase trajectories $\tilde{\mathbf{z}}(t_i)$ from the observations $\tilde{\mathbf{x}}(t_i)$. The $\tilde{\mathbf{z}}(t_i)$ will be the noisy estimates of the $\mathbf{z}(t_i)$. The reconstruction will be based on building an embedding of the manifold M. In general, an embedding of a manifold M is a smooth map $\phi : M \to N$ into another manifold N that is a diffeomorphism onto its image $\phi(M)$. It follows that phase trajectories $\mathbf{z}(t) \subset M$ and $\phi(\mathbf{z}(t)) \subset \phi(M)$ are basically identical (more strictly, diffeomorphic). One approach to build an embedding from the time series is the Whitney Embedding Theorem [22]. If α is a generic map from M to $\mathbb{R}^{2m+1}$ then α itself is an embedding. Therefore, the time series $\tilde{\mathbf{x}}(t_i)$ are the reconstructed $\tilde{\mathbf{z}}(t_i)$. However, this approach requires an excessive $2m+1$ number of time series even when m is rather small. Another approach is the Takens embedding theorem [23]. It requires only one observed time series $x(t_i)$. If α is a generic map from M to $\mathbb{R}$ then the embedding is given by the map $\phi(\mathbf{z}) = \big(\alpha(\mathbf{z}), f^{-1}(\alpha(\mathbf{z})), \dots, f^{-k}(\alpha(\mathbf{z}))\big)$ where $k > 2m$. If we apply ϕ to the $\mathbf{z}(t_i)$ we obtain $\phi(\mathbf{z}(t_i)) = \big(x(t_i), x(t_{i-1}), \dots, x(t_{i-k})\big)$ which we call *delay vector* and denote as $\overleftarrow{\mathbf{x}}(t_i)$. If we compose a delay vector from the noisy time series $\tilde{x}(t_i)$ then it will be the reconstructed $\tilde{\mathbf{z}}(t_i)$. Illustration of the Takens theorem is on the Fig. 1.

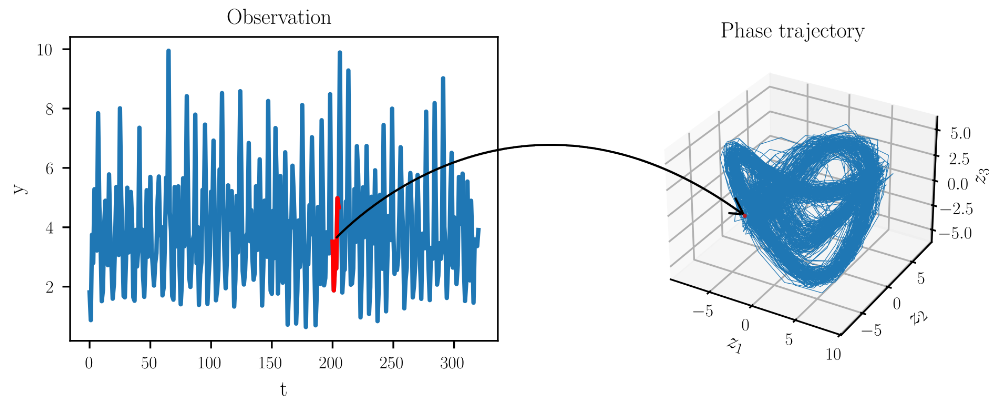

Figure 1 Application of the Takens theorem. On the left are observations $x(t)$. To build a point of the latent phase trajectory $\mathbf{z}(t)$ we must take several lagged observation values (colored in red), e.g. $\mathbf{z}(t_i) = \big(x(t_i), x(t_{i-1}), x(t_{i-2})\big)^{\mathrm{T}}$.

After reconstructing the phase trajectories we will build approximations $\hat{f}_j$ for the system's vector fields f_j. Our approach for the approximation is the Neural Ordinary Differential Equations (NODE) [24]. It allows to use gradient optimization when an optimized objective depends on ODE trajectories. There are plenty of papers using NODE for regression tasks. For example, NODE are used in the context of the physics-informed ODE [25, 26] and control learning [27, 28]. However, very few papers analyze NODE in the context of time series classification. Authors of [29] reinterpreted convolutional neural networks in terms of latent ODE dynamics and used it for hyperspectral image classification. Paper [30] solved classification problem in terms of control theory and NODE. Our paper contributes to the NODE applications for the general dynamical systems.

Actually, the NODE paper [24] introduced its own time series model called latent ODE. It is illustrated on the Fig. 2. The model is probabilistic and is described as

$$\begin{gathered}
\mathbf{z}_{t_0} \sim p_0(\mathbf{z}_{t_0}), \\
\mathbf{z}_{t_1}, \mathbf{z}_{t_2}, \dots, \mathbf{z}_{t_N} = \text{ODESolve}(\mathbf{z}_{t_0}, f, t_0, \dots, t_N), \\
\text{each } \mathbf{x}_{t_i} \sim p_x(x|\mathbf{z}_{t_i})
\end{gathered}$$

where ODESolve() is the solution of the initial value problem. Authors then used a variational autoencoder (VAE) [31] to inference the distributions p_0, p_x and the vector field f. While this model is more general and is widely used [32–34], it has several drawbacks compared to ours. First, it has to parameterize f, p_x and the VAE. Hence, more data is needed in general to obtain good approximations of these functions. We only need to parameterize f. Second, the authors use VAE to obtain $\mathbf{z}(t_0)$ that does not guarantee good approximation [**?**]. We can reconstruct $\mathbf{z}(t_0)$ using the general results discussed above. Third, in the NODE model a connection between the phase trajectories and the observed time series is given by the general distribution p_x. Thus, the connection is difficult to interpret. In addition, the VAE is needed to reconstuct the $\mathbf{z}(t_i)$ from the $\mathbf{x}(t_i)$. Our reconstruction approaches give explicit and easy ways to obtain $\mathbf{z}(t_i)$ from the $\mathbf{x}(t_i)$ and vice versa. Several papers using Takens theorem ideas with the NODE to restore latent dynamics can be found in the literature [35, 36]. But unfortunately, it is poorly covered for machine learning applications. Our paper aims to close this gap.

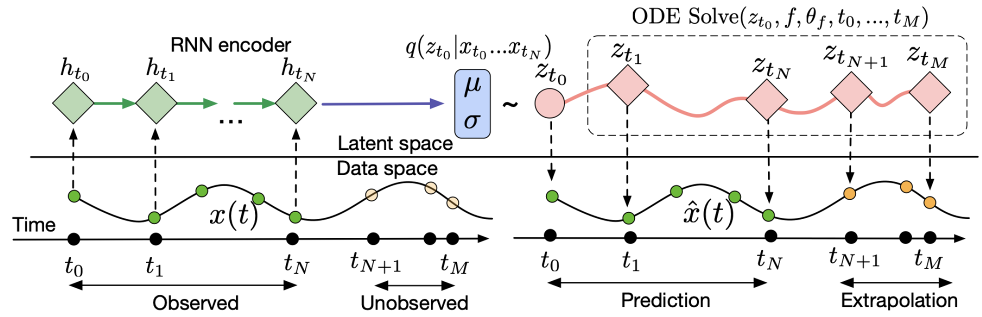

Figure 2 Computational graph of the latent ODE model. Source: reprinted from Chen et al.(2018)

After obtaining field approximations $\hat{f}_j$ we will build a classifier. It will be based on solving the IVP for every $\hat{f}_j$. The full algorithm will be described in the next section. After, we will point out 4 sources of errors influencing classifier's performance. We call such classifier *unrobust*. We will give theoretical estimation of the accuracy decrease due to the one of the errors type. Then we will propose a simple solution to the problem called *trajectory slicing*. We will analyze its advantages and trade-offs. Such a classifier will be *robust*. Next, we will demonstrate all problems of the unrobust classifier and how trajectory slicing solves them in the empirical experiment. The experiment will involve simple two-dimensional systems. Finally, we will apply our approach to the real-world systems. The experiments will include time series from an Inertial Measurement Unit (IMU), atmospheric measurements, ...TODO.

2 Classification algorithm

General classification algorithm can be formulated in the following way:

1. Choose vector field approximation set $\hat{f} \in \mathcal{F}$, vector field loss function $\mathcal{L}(\tilde{\mathbf{z}}(t_i)|\hat{f})$ and trajectory distance function $\rho(\mathbf{z}_1(t_i), \mathbf{z}_2(t_i))$.
2. For each class $y \in 1 \dots K$ denote a number of labeled series as N_i, so $N_1 + \dots + N_K = N$. To find vector field approximation for class y solve

$$\sum_{j=0}^{N_i} \mathcal{L}(\tilde{\mathbf{z}}_j(t_i)|\hat{f}_j) \to \min_{\hat{f}_j \in \mathcal{F}} \tag{2}$$

3.

3 Trajectory slicing

4 Empirical study

References

[1] Berndt, D.J., Clifford, J.: Using dynamic time warping to find patterns in time series. In: Proceedings of the 3rd International Conference on Knowledge Discovery and Data Mining. AAAIWS'94, pp. 359–370. AAAI Press, ??? (1994)

[2] Ding, H., Trajcevski, G., Scheuermann, P., Wang, X., Keogh, E.: Querying and mining of time series data: experimental comparison of representations and distance measures. Proc. VLDB Endow. **1**(2), 1542–1552 (2008) https://doi.org/10.14778/1454159.1454226

[3] Nanopoulos, A., Alcock, R., Manolopoulos, Y.: Feature-based classification of time-series data, pp. 49–61. Nova Science Publishers, Inc., USA (2001)

[4] Wang, X., Smith, K., Hyndman, R.: Characteristic-based clustering for time series data. Data Min. Knowl. Discov. **13**(3), 335–364 (2006) https://doi.org/10.1007/s10618-005-0039-x

[5] Mörchen, F.: Time series feature extraction for data mining using dwt and dft (2003)

[6] Rajpurkar, P., Hannun, A., Haghpanahi, M., Bourn, C., Ng, A.: Cardiologist-level arrhythmia detection with convolutional neural networks (2017) https://doi.org/10.48550/arXiv.1707.01836

[7] Lawhern, V., Solon, A., Waytowich, N., Gordon, S., Hung, C., Lance, B.: Eegnet: a compact convolutional neural network for eeg-based brain–computer interfaces. Journal of Neural Engineering **15** (2018) https://doi.org/10.1088/1741-2552/aace8c

[8] Malhotra, P., Ramakrishnan, A., Anand, G., Vig, L., Agarwal, P., Shroff, G.: Lstm-based encoder-decoder for multi-sensor anomaly detection (2016) https://doi.org/10.48550/arXiv.1607.00148

[9] Trigeorgis, G., Ringeval, F., Brueckner, R., Marchi, E., Nicolaou, M.A., Schuller, B., Zafeiriou, S.: Adieu features? end-to-end speech emotion recognition using a deep convolutional recurrent network. In: 2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 5200–5204 (2016). https://doi.org/10.1109/ICASSP.2016.7472669

[10] Hammerla, N., Halloran, S., Ploetz, T.: Deep, convolutional, and recurrent models for human activity recognition using wearables. (2016)

[11] Shi, X., Chen, Z., Wang, H., Yeung, D.-Y., Wong, W.K., WOO, W.-c.: Convolutional lstm network: A machine learning approach for precipitation nowcasting (2015)

[12] Bifurcation of Fixed Points of Vector Fields, pp. 356–497. Springer, New York, NY (2003). https://doi.org/10.1007/0-387-21749-5_21 . https://doi.org/10.1007/

0-387-21749-5_21

[13] Buzsáki, G., Anastassiou, C., Koch, C.: The origin of extracellular fields and currents—eeg, ecog, lfp and spikes. Nature reviews. Neuroscience **13**, 407–20 (2012) https://doi.org/10.1038/nrn3241

[14] Daunizeau, J., Friston, K.J., Kiebel, S.J.: Variational bayesian identification and prediction of stochastic nonlinear dynamic causal models. Physica D: Nonlinear Phenomena **238**(21), 2089–2118 (2009) https://doi.org/10.1016/j.physd.2009.08.002

[15] Kalnay, E.: Atmospheric Modeling, Data Assimilation and Predictability. Cambridge University Press, ??? (2002)

[16] Michelangeli, P.-A., Vautard, R., Legras, B.: Weather regimes: Recurrence and quasi stationarity. J. Atmos. Sci. **52**, 1237–1256 (1995) https://doi.org/10.1175/1520-0469(1995)052⟨1237:WRRAQS⟩2.0.CO;2

[17] Wheeler, M., Hendon, H.: An all-season real-time multivariate mjo index: Development of an index for monitoring and prediction. Monthly Weather Review - MON WEATHER REV **132**, 1917–1932 (2004) https://doi.org/10.1175/1520-0493(2004)132⟨1917:AARMMI⟩2.0.CO;2

[18] Zhao, J., Li, X., Shum, C., McPhee, J.: A review of physics-based and data-driven models for real-time control of polymer electrolyte membrane fuel cells. Energy and AI **6**, 100114 (2021) https://doi.org/10.1016/j.egyai.2021.100114

[19] Anguita, D., Ghio, A., Oneto, L., Parra, X., Reyes-Ortiz, J.L.: A public domain dataset for human activity recognition using smartphones. In: The European Symposium on Artificial Neural Networks (2013). https://api.semanticscholar.org/CorpusID:6975432

[20] Mekruksavanich, S., Jitpattanakul, A.: Deep convolutional neural network with rnns for complex activity recognition using wrist-worn wearable sensor data. Electronics **10**, 1685 (2021) https://doi.org/10.3390/electronics10141685

[21] Igual, R., Medrano, C., Plaza, I.: Challenges, issues and trends in fall detection systems. Biomedical engineering online **12**, 66 (2013) https://doi.org/10.1186/1475-925X-12-66

[22] Whitney, H.: Differentiable manifolds. Annals of Mathematics **37**(3), 645–680 (1936). Accessed 2025-11-09

[23] Takens, F.: Detecting strange attractors in turbulence. In: Dynamical Systems and Turbulence, Warwick 1980: Proceedings of a Symposium Held at the University of Warwick 1979/80, pp. 366–381 (2006). Springer

[24] Chen, R.T.Q., Rubanova, Y., Bettencourt, J., Duvenaud, D.: Neural ordinary differential equations. In: Proceedings of the 32nd International Conference on Neural Information Processing Systems. NIPS'18, pp. 6572–6583. Curran Associates Inc., Red Hook, NY, USA (2018)

[25] Lai, Z., Mylonas, C., Nagarajaiah, S., Chatzi, E.: Structural identification with physics-informed neural ordinary differential equations. Journal of Sound and Vibration **508**, 116196 (2021) https://doi.org/10.1016/j.jsv.2021.116196

[26] Lai, Z., Liu, W., Jian, X., Bacsa, K., Sun, L., Chatzi, E.: Neural Modal ODEs: Integrating Physics-based Modeling with Neural ODEs for Modeling High Dimensional Monitored Structures. https://doi.org/10.48550/arXiv.2207.07883

[27] Yildiz, C., Heinonen, M., Lahdesmaki, H.: In: Advances in Neural Information Processing Systems

[28] Du, J., Futoma, J., Doshi-Velez, F.: Model-based reinforcement learning for semi-markov decision processes with neural odes. NIPS '20. Curran Associates Inc., Red Hook, NY, USA (2020)

[29] Paoletti, M.E., Haut, J.M., Plaza, J., Plaza, A.: Neural ordinary differential equations for hyperspectral image classification. IEEE Transactions on Geoscience and Remote Sensing **58**(3), 1718–1734 (2020) https://doi.org/10.1109/TGRS.2019.2948031

[30] Ruiz-Balet, D., Zuazua, E.: Neural ode control for classification, approximation, and transport. SIAM Review **65**(3), 735–773 (2023) https://doi.org/10.1137/21M1411433 https://doi.org/10.1137/21M1411433

[31] Kingma, D.P., Welling, M.: Auto-encoding variational bayes. CoRR **abs/1312.6114** (2013)

[32] Kidger, P., Morrill, J., Foster, J., Lyons, T.: Neural controlled differential equations for irregular time series. In: Proceedings of the 34th International Conference on Neural Information Processing Systems. NIPS '20. Curran Associates Inc., Red Hook, NY, USA (2020)

[33] Herrera, C., Krach, F., Teichmann, J.: Neural jump ordinary differential equations: Consistent continuous-time prediction and filtering. arXiv: Machine Learning (2020)

[34] Seedat, N., Imrie, F., Bellot, A., Qian, Z., Schaar, M.: Continuous-time modeling of counterfactual outcomes using neural controlled differential equations. In: International Conference on Machine Learning (2022). https://api.semanticscholar.org/CorpusID:249712326

[35] Ouala, S., Fablet, R., Drumetz, L., Chapron, B., Pascual, A., Collard, F.,

Gaultier, L.: Physically informed neural networks for the simulation and data-assimilation of geophysical dynamics, pp. 3490–3493 (2020). https://doi.org/10.1109/IGARSS39084.2020.9323081

[36] Ouala, S., Nguyen, D., Drumetz, L., Chapron, B., Pascual, A., Collard, F., Gaultier, L., Fablet, R.: Learning latent dynamics for partially observed chaotic systems. Chaos: An Interdisciplinary Journal of Nonlinear Science **30**, 103121 (2020) https://doi.org/10.1063/5.0019309

[37] Bishop, C.M.: Pattern Recognition and Machine Learning (Information Science and Statistics). Springer, Berlin, Heidelberg (2006)

[38] Hogg, R.V., McKean, J.W., Craig, A.T.: Introduction to Mathematical Statistics. Pearson education international. Pearson Education, ??? (2005)

[39] Hairer, E., Norsett, S., Wanner, G.: Solving Ordinary Differential Equations I: Nonstiff Problems vol. 8, (1993). https://doi.org/10.1007/978-3-540-78862-1